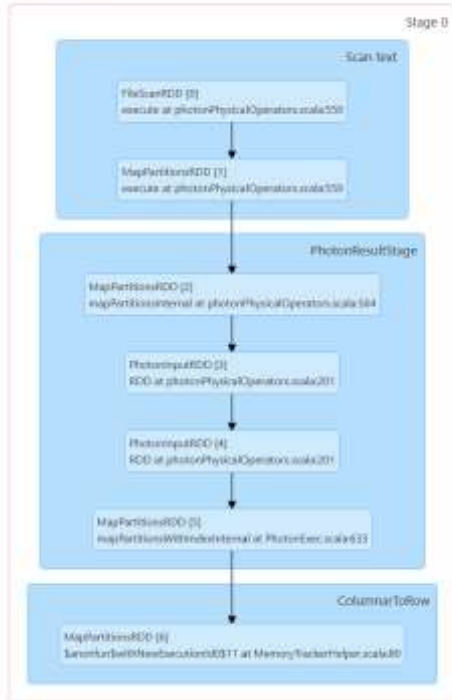


Jobs

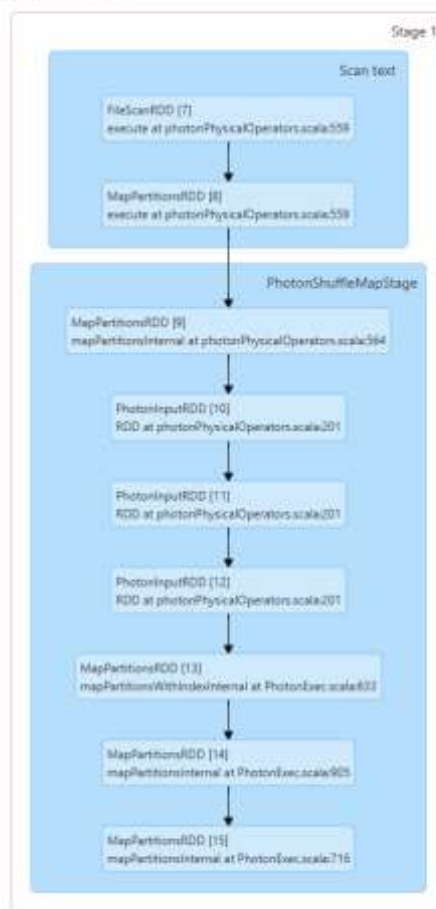
Associated Job Ids: 0

▼ DAG Visualization



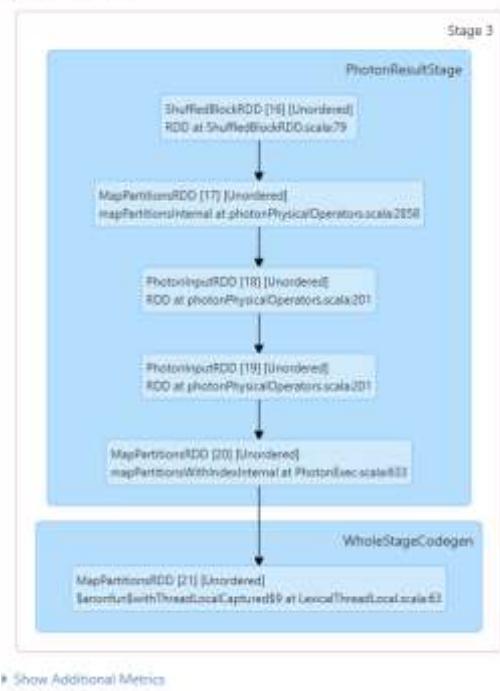
Associated Job Ids: 1

▼ DAG Visualization



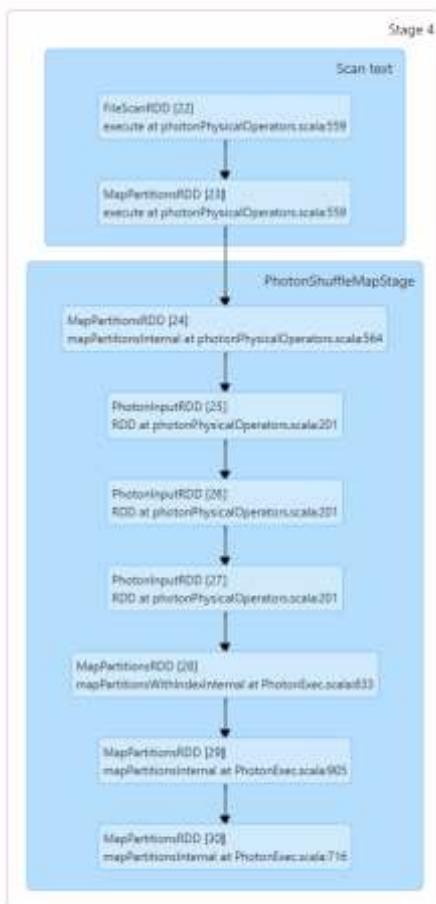
Associated Job Ids: 2

▼ DAG Visualization



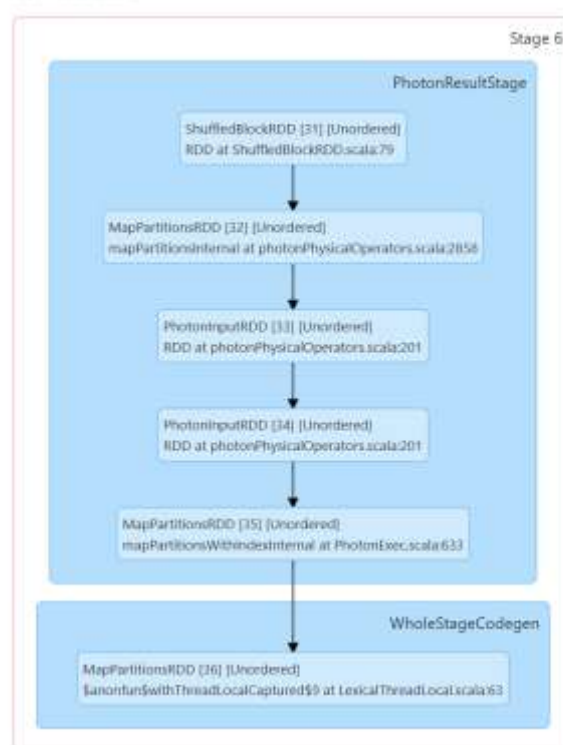
Associated Job Ids: 3

▼ DAG Visualization



Associated Job Ids: 4

▼ DAG Visualization



Spark utworzył osobne Jobs dla każdej akcji (show(), count(), printSchema()). Transformacje takie jak withColumn(), groupBy() czy pivot() nie uruchamiały obliczeń od razu, ponieważ Spark stosuje mechanizm leniwego wykonywania (lazy evaluation). Dopiero wywołanie akcji powodowało uruchomienie odpowiedniego Job.

Job 0 – odczyt pliku FASTQ oraz przygotowanie danych do wyświetlenia (show()). W DAG widoczny jest etap Scan text, czyli odczyt pliku tekstowego oraz przygotowanie danych do prezentacji.

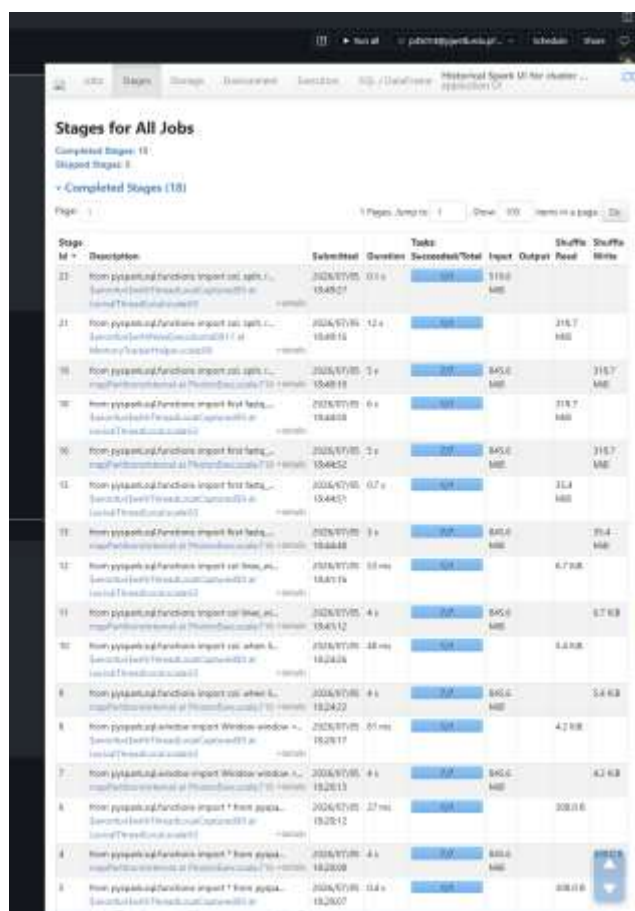
Job 1 – W DAG pojawia się PhotonShuffleMapStage, co oznacza konieczność przetasowania (shuffle) danych pomiędzy partycjami.

Job 2 – wykonanie operacji agregujących. Widoczny jest ShuffledBlockRDD, który potwierdza wykorzystanie danych po operacji shuffle.

Job 3 – kolejne przekształcenia wymagające wymiany danych między partycjami. Struktura DAG jest podobna do Job 1 i również zawiera PhotonShuffleMapStage.

Job 4 – końcowe przetwarzanie danych po operacji pivot() i przygotowanie wyników do wyświetlenia. W DAG ponownie pojawia się ShuffledBlockRDD oraz etap WholeStageCodegen, oznaczający optymalizację wykonywaną przez Spark.

Stages



The screenshot shows the 'Stages for All Jobs' section in the Spark UI. It displays a table with 18 completed stages. The table columns include Stage ID, Description, Submitted, Duration, Successes/Total, Input, Output, Peak, Shuffle, and Memory. The stages are listed in descending order of duration.

Stage ID	Description	Submitted	Duration	Successes/Total	Input	Output	Peak	Shuffle	Memory
22	from pySpark.functions import col, split, ...	2026/07/05 15:45:27	0.0 s	1/1	MB	MB	MB	MB	MB
21	from pySpark.functions import col, split, ...	2026/07/05 15:48:15	12 s	1/1	MB	MB	MB	215.7 MB	MB
19	from pySpark.functions import col, split, ...	2026/07/05 15:48:19	5 s	1/1	MB	MB	MB	315.7 MB	MB
20	from pySpark.functions import first, last, ...	2026/07/05 15:48:19	6 s	1/1	MB	MB	MB	215.7 MB	MB
18	from pySpark.functions import first, last, ...	2026/07/05 15:48:52	5 s	1/1	MB	MB	MB	315.7 MB	MB
17	from pySpark.functions import first, last, ...	2026/07/05 15:44:51	0.7 s	1/1	MB	MB	MB	31.4 MB	MB
16	from pySpark.functions import first, last, ...	2026/07/05 15:48:48	3 s	1/1	MB	MB	MB	31.4 MB	MB
15	from pySpark.functions import first, last, ...	2026/07/05 15:47:19	0.5 ms	1/1	MB	MB	MB	6.7 MB	MB
14	from pySpark.functions import col, split, ...	2026/07/05 15:47:12	4 s	1/1	MB	MB	MB	5.7 MB	MB
13	from pySpark.functions import col, split, ...	2026/07/05 15:28:06	18 ms	1/1	MB	MB	MB	5.4 MB	MB
12	from pySpark.functions import col, split, ...	2026/07/05 15:24:22	6 s	1/1	MB	MB	MB	5.4 MB	MB
11	from pySpark.functions import col, split, ...	2026/07/05 15:28:17	0.1 ms	1/1	MB	MB	MB	4.7 MB	MB
10	from pySpark.functions import col, split, ...	2026/07/05 15:28:13	6 s	1/1	MB	MB	MB	4.7 MB	MB
9	from pySpark.functions import first, last, ...	2026/07/05 15:28:12	2.7 ms	1/1	MB	MB	MB	308.0 MB	MB
8	from pySpark.functions import first, last, ...	2026/07/05 15:28:08	4 s	1/1	MB	MB	MB	308.0 MB	MB
7	from pySpark.functions import first, last, ...	2026/07/05 15:28:07	0.8 s	1/1	MB	MB	MB	308.0 MB	MB

W zakładce Stages Spark UI widoczne było:

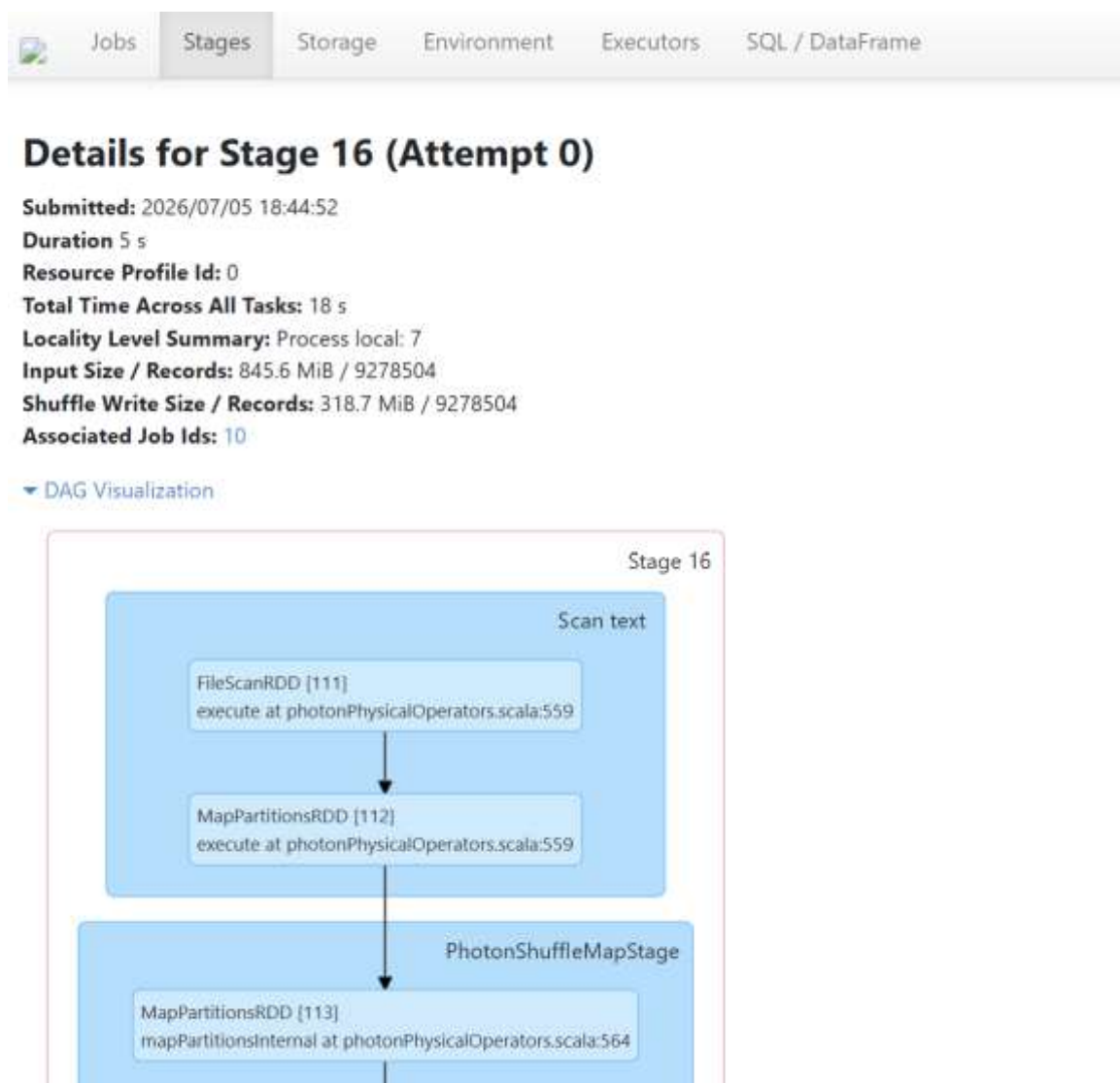
- Completed Stages: 18

- Skipped Stages: 6

Większość prostych operacji (np. `show()` po `withColumn()`) wykonywała się w jednym Stage. Bardziej złożone operacje (`pivot()` i `groupBy()`) wymagały kilku etapów przetwarzania.

Widoczne Skipped Stages oznaczają, że Spark wykorzystał własne mechanizmy optymalizacji i nie wykonywał ponownie etapów, których wynik był już dostępny. Pozwoliło to ograniczyć liczbę niepotrzebnych obliczeń

Locality Level



W szczegółach Stage 19 Spark UI wyświetla:

Locality Level Summary: Process local: 7

Natomiast w tabeli Tasks wszystkie zadania posiadały poziom lokalności:

PROCESS_LOCAL

Oznacza to, że wszystkie 7 zadań zostało wykonanych lokalnie na procesie, który posiadał wymagane dane. Dzięki temu Spark nie musiał przysyłać danych przez sieć pomiędzy węzłami klastra, co zmniejsza opóźnienia i zwiększa wydajność przetwarzania.

Część 2

```
from pyspark.sql.window import Window

window = Window.orderBy(monotonically_increasing_id())

lines_with_id = lines_df.withColumn(
    "line_number",
    row_number().over(window) - 1
)

lines_with_id.show(8, truncate=False)
```

*(1) Spark Job

↳ Job 5: 'lines' (Stages: 2/2)

↳ lines_with_id: pyspark.sql.DataFrame = [value:string,line_number:integer]

value	line_number
[GSRRL6556247.1 1 length=151	0
[GCTCCCAACCAAGCTCTNTTGAAGATCTTGAAGAACTGAATTCAAAAGATCAAAAGCTGGGCTCCGTCGTTGACAGGTATAGGTAAAGTCCCTGACAGGCTCTGAGCTGGGCTGACAGGCTCTCATGCTGCTG	1
[+SRRL6556247.1 1 length=151	2
[AAGATCTTGAAGATCTTGAAGAACTGAATTCAAAAGATCAAAAGCTGGGCTCCGTCGTTGACAGGTATAGGTAAAGTCCCTGACAGGCTCTGAGCTGGGCTGACAGGCTCTCATGCTGCTG	3
[GSRRL6556247.2 2 length=151	4
[GCTCCCAACCAAGCTCTNTTGAAGATCTTGAAGAACTGAATTCAAAAGATCAAAAGCTGGGCTCCGTCGTTGACAGGTATAGGTAAAGTCCCTGACAGGCTCTGAGCTGGGCTGACAGGCTCTCATGCTGCTG	5
[+SRRL6556247.2 2 length=151	6
[AAGATCTTGAAGATCTTGAAGAACTGAATTCAAAAGATCAAAAGCTGGGCTCCGTCGTTGACAGGTATAGGTAAAGTCCCTGACAGGCTCTGAGCTGGGCTGACAGGCTCTCATGCTGCTG	7

only showing top 8 rows

Część 3

```
from pyspark.sql.functions import col, when

lines_typed = lines_with_id.withColumn(
    "line_type",
    when(col("line_number") % 4 == 0, "header")
    .when(col("line_number") % 4 == 1, "sequence")
    .when(col("line_number") % 4 == 2, "separator")
    .when(col("line_number") % 4 == 3, "quality")
)

lines_typed.show(12, truncate=False)
```

*(1) Spark Job

↳ lines_typed: pyspark.sql.DataFrame = [value:string,line_number:integer,... 1 more field]

value	line_number	line_type
[GSRRL6556247.1 1 length=151	0	header
[GCTCCCAACCAAGCTCTNTTGAAGATCTTGAAGAACTGAATTCAAAAGATCAAAAGCTGGGCTCCGTCGTTGACAGGTATAGGTAAAGTCCCTGACAGGCTCTGAGCTGGGCTGACAGGCTCTCATGCTGCTG	1	sequence
[+SRRL6556247.1 1 length=151	2	separator
[AAGATCTTGAAGATCTTGAAGAACTGAATTCAAAAGATCAAAAGCTGGGCTCCGTCGTTGACAGGTATAGGTAAAGTCCCTGACAGGCTCTGAGCTGGGCTGACAGGCTCTCATGCTGCTG	3	quality
[GSRRL6556247.2 2 length=151	4	header
[GCTCCCAACCAAGCTCTNTTGAAGATCTTGAAGAACTGAATTCAAAAGATCAAAAGCTGGGCTCCGTCGTTGACAGGTATAGGTAAAGTCCCTGACAGGCTCTGAGCTGGGCTGACAGGCTCTCATGCTGCTG	5	sequence
[+SRRL6556247.2 2 length=151	6	separator
[AAGATCTTGAAGATCTTGAAGAACTGAATTCAAAAGATCAAAAGCTGGGCTCCGTCGTTGACAGGTATAGGTAAAGTCCCTGACAGGCTCTGAGCTGGGCTGACAGGCTCTCATGCTGCTG	7	quality
[GSRRL6556247.3 3 length=151	8	header
[GCTCCCAACCAAGCTCTNTTGAAGATCTTGAAGAACTGAATTCAAAAGATCAAAAGCTGGGCTCCGTCGTTGACAGGTATAGGTAAAGTCCCTGACAGGCTCTGAGCTGGGCTGACAGGCTCTCATGCTGCTG	9	sequence
[+SRRL6556247.3 3 length=151	10	separator
[AAGATCTTGAAGATCTTGAAGAACTGAATTCAAAAGATCAAAAGCTGGGCTCCGTCGTTGACAGGTATAGGTAAAGTCCCTGACAGGCTCTGAGCTGGGCTGACAGGCTCTCATGCTGCTG	11	quality

only showing top 12 rows

Część 4

[illegible][illegible]

Część 5

[illegible]

Część 6

[illegible]